



QWASAR
Silicon Valley

Qwasar Master's of Science in Computer Science Programs
Copyright 2026
Engineering Lab
Marry Me



Problem Statement

Your best friend is getting married and put you in charge of coordination. Build a small event-driven simulation that receives wedding incidents, routes them to the right team, and tracks guest stress when events go unhandled.

Objective

Design and implement a simulation that receives events, dispatches them to teams based on type, and handles them with available workers — all within time constraints.

Solution Overview

1. Coordinator — Receives and validates incoming events, then forwards them to the appropriate team.
2. Teams — Each team has a fixed pool of workers. When an event arrives, the team assigns an idle worker. If no worker is available, the event waits in a queue.
3. Stress Tracking — If an event is not handled before its priority deadline expires, it is discarded and the global stress level increments by 1.

Entities

Workers

- current_status: Idle | Working
- Handling an event takes 3 seconds, then the worker returns to Idle.

Team	Handles Event Type
Security	brawl, not_on_list
Catering	Bad_food, feeling_ill
Waiters	Dirty_table, broken_item

Events

```
JSON
event {
  id: int,
  event_type: string,           // must match a known type above
  priority: high | medium | low,
  description: string,
  timestamp: float              // seconds since simulation start
}
```

Priority deadlines (from event timestamp):

Priority	Deadline
High	5 seconds
Medium	10 seconds
Low	15 seconds

Simulation

- The simulation runs for 60 seconds.
- Events are provided as a JSON array (input file), each with a timestamp indicating when it arrives.
- At the end, print:
 - Total events received
 - Total events handled
 - Total events expired
 - Final stress level (= number of expired events)

Example Input (events.json)

JSON

```
[
  {"id": 1, "event_type": "brawl", "priority": "high", "description": "fight near the bar", "timestamp": 2.0},
  {"id": 2, "event_type": "bad_food", "priority": "medium", "description": "cold soup", "timestamp": 3.0},
  {"id": 3, "event_type": "dirty_table", "priority": "low", "description": "table 5 is a mess", "timestamp": 4.0},
  {"id": 4, "event_type": "brawl", "priority": "high", "description": "another fight", "timestamp": 5.0},
  {"id": 5, "event_type": "feeling_ill", "priority": "high", "description": "guest fainted", "timestamp": 5.5}
]
```

Constraints

- Language: Python, Go, Node.js, Java, Ruby, or Rust.
- No external message brokers required — in-process queues are fine.
- Events must be processed asynchronously (use threads, goroutines, async/await, etc.).
- Invalid event_type values should be logged and skipped.

Deliverables

1. Source code in a repository with build/run instructions.
2. Console output showing: event received/dispatched/handled/expired log lines, and the final stress summary.

Learning Outcomes

- Asynchronous event dispatching and routing by type.
- Worker pool management with concurrency primitives.
- Time-based expiration logic under constrained resources.